

LangChain

Day – 11

1. What is LangChain?

LangChain is a framework for developing applications with large language models (LLMs). It helps you chain together prompts, LLMs, APIs, and other data sources to build powerful AI applications like chatbots, summarizers, question-answering systems, and agents.

Key Points:

- Designed for AI developers and prompt engineers.
- Makes complex multi-step workflows easier with LLMs.
- Enables integration with external tools, databases, and APIs.

2. Why LangChain is Important in Prompt Engineering?

Prompt engineering is about crafting effective prompts for LLMs. LangChain adds another layer:

1. Prompt Templates: Reuse prompts and make them dynamic with variables.
2. Chaining: Combine multiple LLM calls in sequence (e.g., summarize → translate → generate questions).
3. Memory: Store user inputs and context across multiple interactions.
4. Agents: Let LLMs decide actions dynamically based on prompts and available tools.

Example:

- Prompt: "Summarize this text."
- LangChain can automate fetching the text → summarizing → storing summary → generating a report.

3. Advantages of Using LangChain in Prompt Engineering

- **Scalability:** Build complex AI workflows without manual coding.
- **Reusability:** Templates and chains can be reused across projects.
- **Integration:** Works with APIs, databases, and external tools.
- **Dynamic Responses:** Can generate context-aware outputs.

Code

```
from langchain_google_genai import ChatGoogleGenerativeAI

import os

# ✅ Set your Gemini API Key
os.environ["GOOGLE_API_KEY"] = "AI.....HA"

# ✅ Initialize Gemini Chat Model (via LangChain)
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.7)

# ✅ Simple Prompt
prompt = "Explain JWT tokens?."

# ✅ Get Response
response = llm.invoke(prompt)

# ✅ Output
print("🤖 Gemini AI says:\n", response.content)
```

****How JWTs Work in Practice:****

1. ****Authentication:**** The user provides their credentials (username and password) to the server.
2. ****Verification:**** The server verifies the credentials against its user database.
3. ****Token Generation:**** If the credentials are valid, the server generates a JWT containing claims about the user (e.g., user ID, roles, permissions). The JWT is signed using a secret key known only to the server.
4. ****Token Issuance:**** The server returns the JWT to the client.
5. ****Storage:**** The client stores the JWT (typically in local storage, cookies, or session storage).
6. ****Authorization:**** For every subsequent request to protected resources, the client includes the JWT in the `Authorization` header (usually with the `Bearer` scheme):

 ```  
 Authorization: Bearer <JWT>  
 ```
7. ****Verification:**** The server receives the request and extracts the JWT from the `Authorization` header.
8. ****Signature Verification:**** The server verifies the JWT's signature using its secret key. If the signature is invalid, the token has been tampered with, and the request is rejected.
9. ****Claims Validation:**** The server validates the claims in the JWT, such as the expiration time (`exp`), issuer (`iss`), and audience (`aud`). If the claims are invalid, the request is rejected.
10. ****Authorization:**** If the signature and claims are valid, the server trusts the JWT and grants access to the protected resource based on the information contained in the claims (e.g., user ID, roles, permissions).

✅ Summary

- You **connect to Google Gemini LLM via LangChain**, send a prompt, and get a response.
- The flow is: **Set API key → Initialize model → Define prompt → Invoke → Print response.**

- This code is a **basic template for prompt engineering** where you can experiment with different prompts, temperature values, or model types.

4. Chains in LangChain

1. What is a Chain?

A **Chain** in LangChain is a **sequence of actions or calls** that are executed in order, usually involving one or more **LLMs, prompts, or tools**.

Think of it as a **pipeline**:

- Input → Step 1 → Step 2 → Step 3 → Output

Why Chains Are Important?

- **Automation:** Execute multi-step tasks automatically.
- **Modularity:** Each chain step can be reused.
- **Complex Workflows:** Handle tasks like summarizing → translating → generating questions in one chain.
- **Prompt Engineering:** Test and combine different prompts easily.

Example:

```
from langchain import LLMChain, OpenAI
from langchain.prompts import PromptTemplate

llm = OpenAI(temperature=0.5)

template = "Explain {topic} in simple words."
prompt = PromptTemplate(template=template, input_variables=["topic"])

chain = LLMChain(llm=llm, prompt=prompt)

result = chain.run(topic="JWT tokens")

print(result)
```